# Training and Optimizing Deep Neural Networks

## Deep Neural Networks Intro and Architecture

### Introduction

A **deep neural network (DNN)** is a machine learning model made of many simple units called **neurons**, arranged in **layers**, that learn to map inputs (like pixels, words, data) to outputs (like labels, predictions, decisions).

- **"Neural"** because it's loosely inspired by the brain.
- **"Network"** because neurons are connected in a graph of layers.
- **"Deep"** because it has **multiple hidden layers**, not just one.

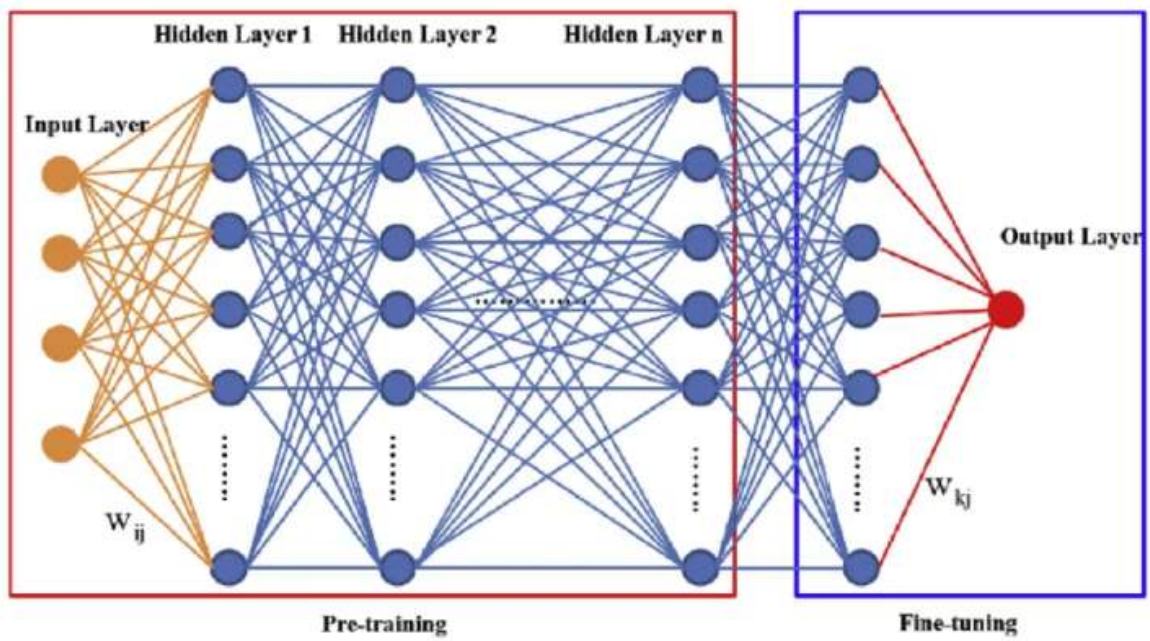



Figure 1. Structural diagram of deep neural networks.

https://www.researchgate.net/figure/Structural-diagram-of-deep-neural-networks_fig2_340748945

In terms of the big picture: **output=ƒ(input;parameters)**

where the **parameters** are the **weights** and **biases** that the network learns from data.

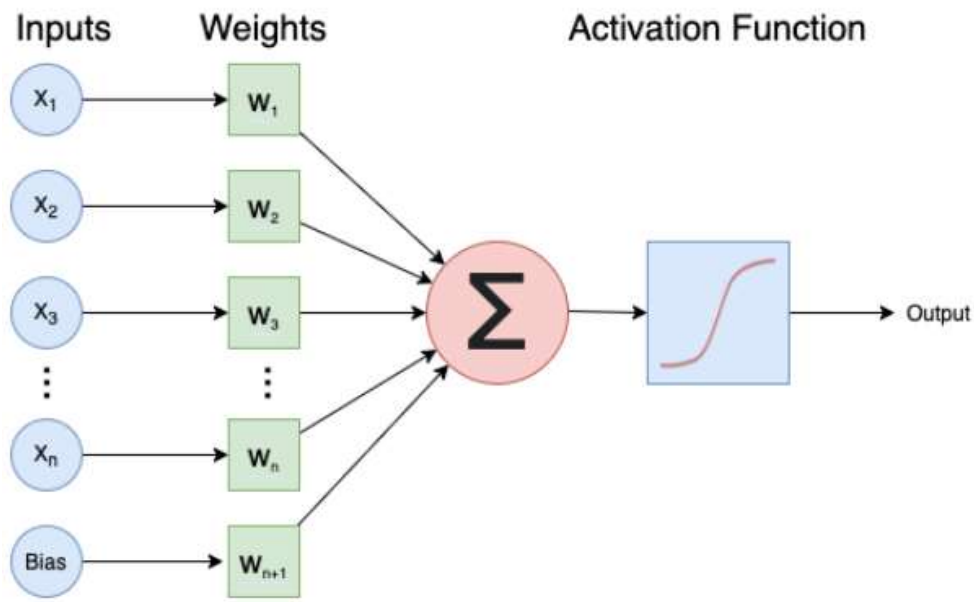



Figure 2. Diagram of a single artificial neuron combining inputs and weights using an activation function

https://uw-madison-datascience.github.io/deep-learning-intro/images.html

A neuron takes some n numbers, combines them, and outputs a number.

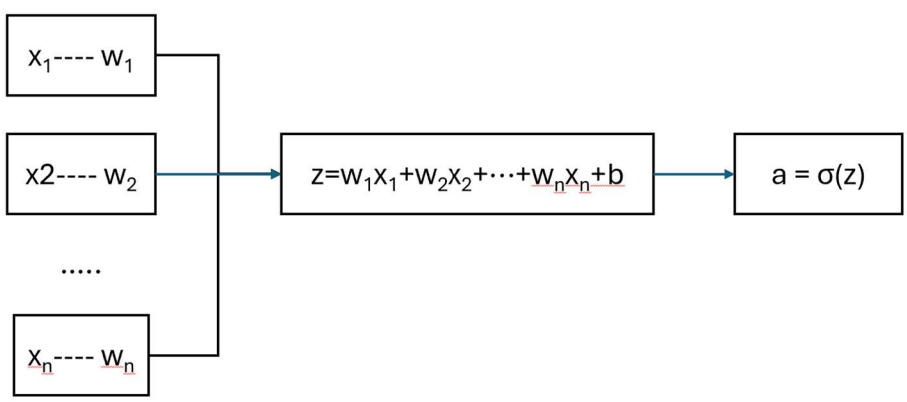
**Inputs:** $x_1, x_2, \ldots, x$.

**Weights:** $w_1, w_2, \ldots, w_n$

Bias: b

The neuron computes a **weighted sum:** $\mathbf{z=w_1x_1+w_2x_2+\cdots+w_nx_n+b}$

Then it passes z through an **activation function** $\sigma(\cdot)$:output=a=$\sigma(z)$



## Activation Functions

An **activation function** is a mathematical function inside each neuron of a neural network that decides whether a neuron should "activate" and how strongly.

Without activation functions, a neural network would behave like a plain linear equation, no matter how many layers we add. With activation functions, the network can learn **complex, nonlinear patterns** like images, speech, and language.

Common activation functions:

| Activation | Formula | Value | Output Range | Good For | Notes |
|---|---|---|---|---|---|
| ReLU (Rectified Linear Unit) | ReLU(z)=max(0,z) | If the signal is positive → pass it through<br><br>if negative → output 0 | 0 to ∞ | Hidden layers | Fast, simple, widely used |
| Sigmoid | $\sigma(z) = \frac{1}{1+e^{-z}}$ | Squashes values between **0 and 1**<br><br>Good for **probabilities** | 0 to 1 | Binary classification output | Can cause vanishing gradients |
| Tanh | $\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$ | Like sigmoid, but outputs between **–1 and 1**<br><br>Zero-centered → often better than sigmoid in hidden layers | –1 to 1 | Hidden layers | Zero-centered, smoother than sigmoid |
| Softmax | $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ | Turns a list of numbers into a **probability distribution**<br><br>All outputs sum to 1 | 0 to 1 (sum=1) | Multi-class output | Produces probabilities |

# DNN Architecture

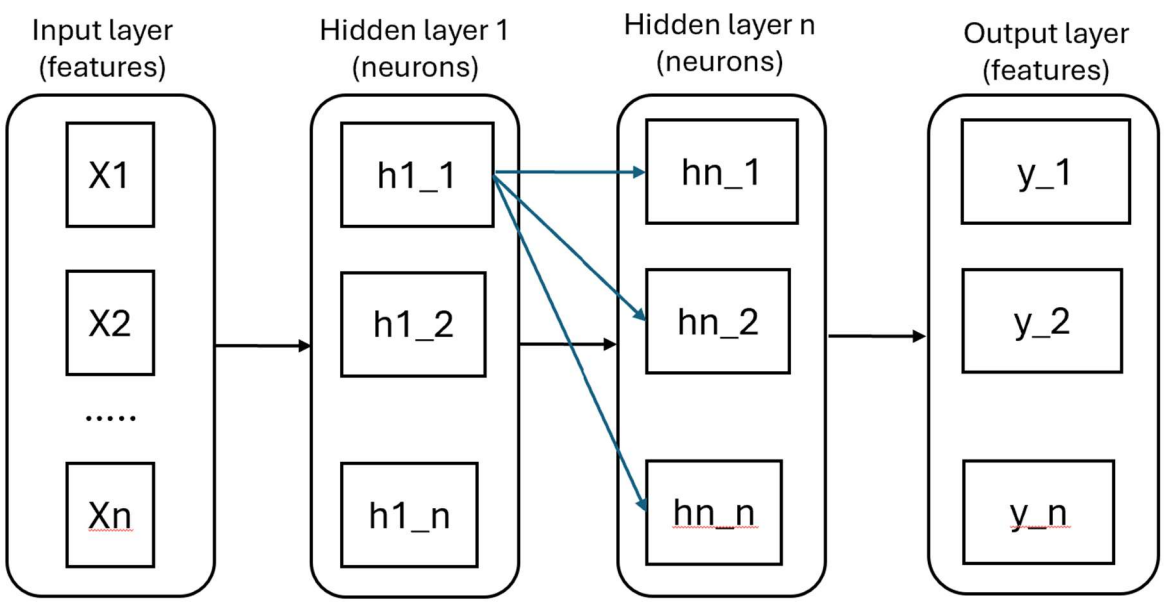
## Layers

A **layer** is a group of neurons that all look at the same input vector.

Types of layers

- **Input layer:**
  - Holds the raw data (e.g., pixel values, features).
  - Usually no computation, just the starting point.
- **Hidden layers:**
  - One or more layers between input and output.
  - Each hidden layer transforms the data into more abstract features.
- **Output layer:**
  - Produces the final prediction (e.g., class probabilities, a number for regression).

A simple feedforward architecture diagram as a classic **fully connected feedforward network**, also called a multilayer perceptron:



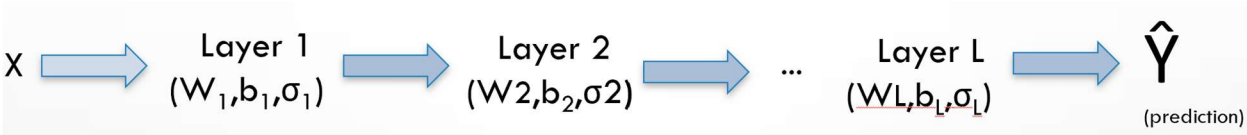

Every neuron in one layer is connected to every neuron in the next layer, from here the terminology fully connected.

## Forward pass: how DNN makes predictions

The **forward pass** is feeding data through the network and making predictions.

1. Start with input x (e.g., image pixels, features).
2. Layer 1**:** $z^{(1)}=W^{(1)}x+b^{(1)}$, $a(1)=\sigma^{(1)}(z^{(1)})$
3. Layer 2**:** $z^{(2)}=W^{(2)a(1)}+b^{(2)}$, $a^{(2)}=\sigma^{(2)}(z(^{2})$
4. Continue until output layer: $\hat{y}=a(L)$

where $\hat{y}$ is the prediction.




## Loss function: measuring how wrong the prediction is

To train the network, we need a way to measure **error** between predictions and true labels. This is done through loss functions.

Common loss functions:

Mean Squared Error (MSE) for regression:

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

Cross-Entropy Loss for classification
Binary:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

Multi-class:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k} y_{i,k}\log(\hat{y}_{i,k})$$

## Backpropagation and learning

Back propagation adjusts the weights to reduce the loss.

How a DNN learns:

1. Initialize weights and biases randomly.
2. For each batch of training data:
   - Do a forward pass to compute predictions ŷ.
   - Compute the loss L.
   - Use backpropagation to compute gradients:

$$\frac{\partial L}{\partial W^{(l)}}, \quad \frac{\partial L}{\partial b^{(l)}}$$

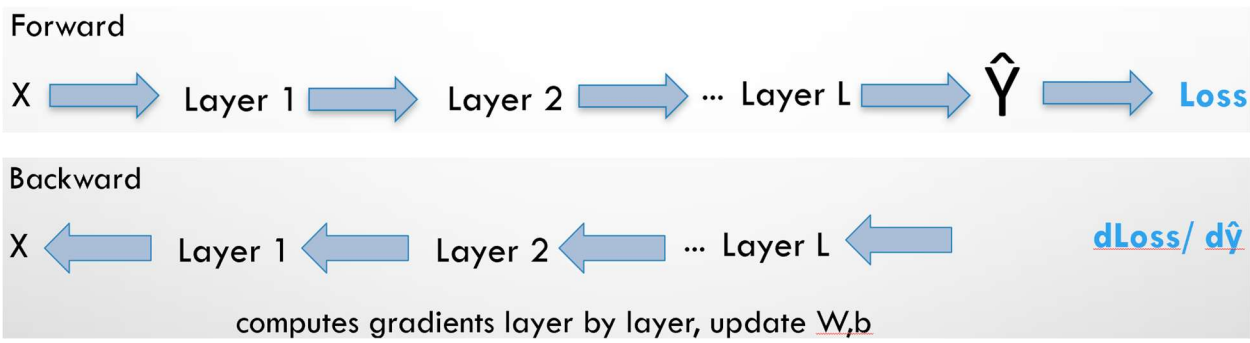
- Update parameters using an **optimizer** (e.g., gradient descent, adam).

Gradient descent update

For a weight w:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

where η is the **learning rate**.




Errors flow backward from the output to earlier layers and each layer learns how to adjust its weights to reduce the loss.

## Deep Networks

A network is called **deep** when it has **multiple hidden layers** (not just one). Depth allows the network to learn **hierarchical representations**:

- In image tasks:
  - Early layers: edges, corners.
  - Middle layers: textures, parts of objects.
  - Deep layers: whole objects (faces, cars, etc.).
- In text tasks:
  - Early layers: word-level patterns.
  - Deeper layers: phrases, sentence meaning, context.

## Common Deep Architectures

### *Fully connected (feedforward) networks*

**Use case:** tabular data, simple tasks.

**Structure:** every neuron in one layer connects to every neuron in the next.

[Input] -> [Dense] -> [Dense] -> [Dense] -> [Output]

Other architectures:

1. Convolutional Neural Networks (CNNs)
2. Recurrent Neural Networks (RNNs) and variants (LSTM, GRU)
3. Transformers (modern deep architecture)

# DNN Hyperparameters Tuning

## Weights Initialization in Deep Neural Networks

Before training begins, every neural network layer needs **initial weights**. These weights determine how the network behaves at the very first forward pass.

Why initialization matters:

- It affects how signals flow through the network
- It affects how gradients flow backward
- Good initialization → stable learning
- Bad initialization → exploding/vanishing gradients, slow training, or poor accuracy

So initialization is not just a detail — it's a **critical part of deep learning**.

There are 2 major initialization strategies:

- **Glorot Uniform** (Xavier Uniform)
    - Default initializer in Keras
    - Used when you don't specify kernel_initializer
    - Glorot tries to keep the **variance of activations** the same across layers
    - Best for: Sigmoid ,Tanh, linear activations, regression tasks, shallow or moderately deep networks, SGD with small learning rates.
- He Initialization (He Normal / He Uniform)
    - Designed specifically for ReLU and its variants
    - Often used in deep CNNs and modern architectures
    - It produces **larger initial weights** than Glorot
    - ReLU "kills" negative values (outputs 0), so He initialization compensates by giving weights **higher variance** to keep activations alive

- Best for: ReLU, LeakyReLU, ELU, deep networks, optimizers like **Adam** or **SGD with momentum,** higher learning rates

## Evaluating Weight Initialization Strategies in a Deep Neural Network

I trained a deep neural network (DNN) to perform the regression task. The primary goal was to evaluate how different **weight initialization strategies** affect model performance when using:

- **ReLU activation functions**
- **Stochastic Gradient Descent (SGD)** as the optimizer
- A small learning rate (0.001)
- A consistent network architecture across experiments

Two models were compared:

1. **Model A:** ReLU activations with the **default initializer** (glorot_uniform)
2. **Model B:** ReLU activations with **He initialization** (he_normal)

Both models were trained for 10 epochs using the same training and validation datasets.

### *Model Architecture*

Both models used the same architecture:

- Dense layer with 64 units, ReLU activation
- Dense layer with 32 units, ReLU activation
- Dense layer with 16 units, ReLU activation
- Output layer with 1 unit (linear activation)

This architecture is typical for regression tasks and allows us to isolate the effect of initialization.

### *Training Configuration*

Both models were compiled with:

- **Loss:** Mean Squared Error (MSE)
- **Metric:** Mean Absolute Error (MAE)
- **Optimizer:** Stochastic Gradient Descent (SGD)
- **Learning rate:** 0.001
- **Epochs:** 10

This setup ensures that any performance differences arise from the initialization strategy rather than other hyperparameters.

## *Results Summary*

| Experiment | Results | Conclusion |
|---|---|---|
| Model A<br><br>ReLU + Glorot Uniform | Starts with **loss = 0.2376**, **val_loss = 0.1935**<br><br>Steady improvement across epochs<br><br>Final metrics:<br><br>• **loss = 0.0443**<br>• **mae = 0.1346**<br>• **val_loss = 0.0668**<br>• **val_mae = 0.1622** | Model A trains smoothly<br><br>Converges well<br><br>Good generalization<br><br>But training starts slower compared to He initialization |
| Model B<br><br>ReLU + He Initialization | Starts with **loss = 0.2447**, **val_loss = 0.1606**<br><br>Faster reduction in validation loss<br><br>Final metrics:<br><br>• **loss = 0.0550**<br>• **mae = 0.1562**<br>• **val_loss = 0.0617**<br>• **val_mae = 0.1751** | Model B converges **faster in early epochs**<br><br>Achieves **lower validation loss** than Model A<br><br>Slightly higher training loss than Model A, but better validation performance<br><br>Indicates **better generalization** |

Model Comparison

| Metric | Model A (Glorot) | Model B (He) | Better |
|---|---|---|---|
| Initial val_loss | 0.1935 | **0.1606** | **Model B** |
| Final val_loss | 0.0668 | **0.0617** | **Model B** |
| Final val_mae | **0.1622** | 0.1751 | Model A (slightly) |
| Convergence speed | Moderate | **Faster** | **Model B** |
| Stability | Smooth | Smooth | Tie |

## *Interpretation*

- He initialization improves early training and final validation loss, which is the most important metric for generalization.
- Glorot performs slightly better on training MAE, but this may indicate mild overfitting.
- Overall, Model B is the stronger model for ReLU networks.

### *Visualization of Learning Curves*

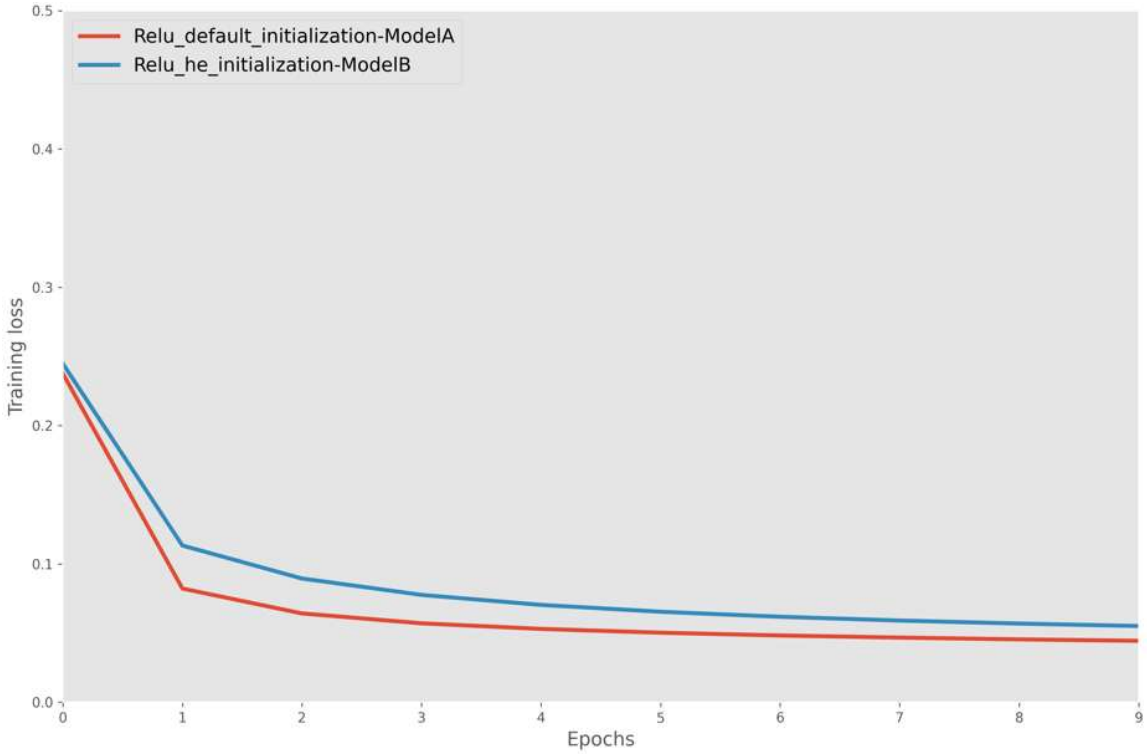



Figure 3: Training Loss

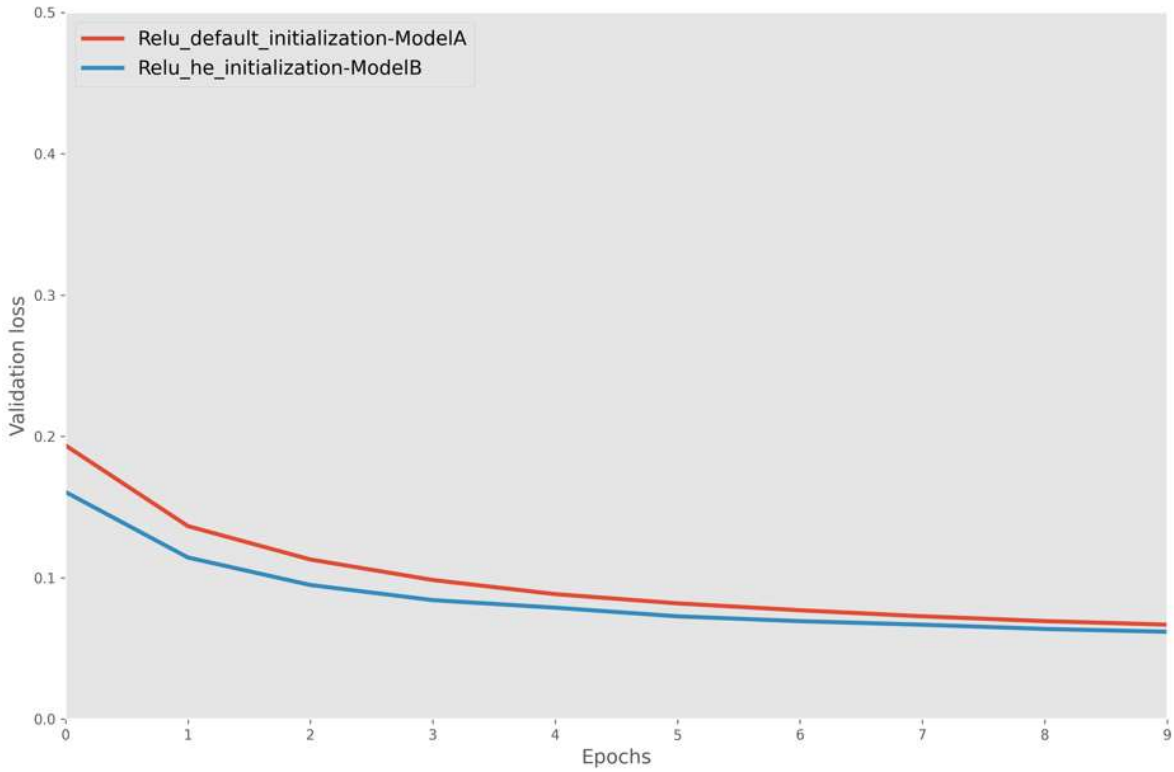



Figure 4: Validation Loss

### *Conclusion*

- Weight initialization has a **significant impact** on training dynamics.
- **He initialization is better suited for ReLU**, leading to:
  - Faster convergence
  - Lower validation loss
  - Better generalization
- Glorot still performs well, but is not optimal for ReLU-based networks.

## Activation Functions in Deep Neural Networks

Activation functions are the **nonlinear transformations** inside a deep neural network that allow it to learn complex patterns like curved, layered, or highly irregular relationships in the data. Without them, a DNN would collapse into a simple linear model, no matter how many layers it has.

Why activation functions matter:

1. They make deep learning possible

Without nonlinear activations, stacking layers would not increase model capacity. The entire network would behave like a single linear transformation.

2. They control gradient flow

The choice of activation affects:

- whether gradients vanish or explode
- how fast the network trains
- how stable optimization is

This is why ReLU and its variants became dominant—they avoid saturation for positive inputs.

## Comparing Activation Functions in a Deep Neural Network

This experiment evaluates how different activation functions affect the training dynamics and predictive performance of a fully connected neural network trained on the same dataset, using the same optimizer (SGD), learning rate, and architecture.

The tested activation functions were:

- ReLU (baseline)
- Leaky ReLU
- ELU
- SELU
- Mixed activations (ReLU + ELU)

All models use the same 4-layer MLP architecture:

- Dense(64)
- Dense(32)
- Dense(16)
- Dense(1)

The goal of the experiment was to understand:

- How activation functions influence convergence speed
- How they affect gradient flow and stability
- Which activation yields the lowest validation error
- Whether mixing activations provides any benefit

### *Summary of Results*

Validation MAE after 10 epochs

| Activation | Final Val MAE | Performance |
|---|---|---|
| ReLU (baseline) | 0.1491 | Good |
| Leaky ReLU | 0.1374 | Very Good |
| **ELU** | **0.1173** | **Best Overall** |
| SELU | 0.1440 | Moderate |
| Mixed (ReLU + ELU) | 0.1502 | Similar to ReLU |

ELU clearly performs the best, achieving the lowest validation MAE and the smoothest convergence.

## *Observations*

- All models improve steadily over 10 epochs.
- Models with smoother negative-side behavior (Leaky ReLU, ELU) converge faster.
- ELU consistently produces the lowest loss and MAE.
- SELU performs well but not as strongly as ELU in this architecture.
- Mixed activations do not outperform pure ELU or Leaky ReLU.

### *Interpretation of Each Activation's Behavior*

| Activation | Behaviour | | Interpretation |
|---|---|---|---|
| ReLU (Baseline) | • Fast computation<br>• Zero output for negative inputs<br>• Can cause "dead neurons" | • | • Performs reasonably well<br>• Slower early learning<br>• Good baseline but not optimal |
| Leaky ReLU | • Allows a small negative slope<br>• Prevents dead neurons<br>• Better gradient flow | • | • Faster convergence than ReLU<br>• Lower final MAE<br>• Strong improvement with minimal complexity |
| ELU (Exponential Linear Unit) | • Smooth negative region<br>• Negative outputs push mean activations toward zero<br>• Helps stabilize gradients | • | • **Best performer overall**<br>• Fastest convergence<br>• Lowest validation MAE<br>• Excellent for noisy or complex regression tasks |
| SELU (Scaled ELU) | • Designed for self-normalizing networks | • | • Good performance but not optimal here |

| | | | |
|---|---|---|---|
| | • Works best with specific architectures (e.g., deep networks with AlphaDropout) | | • Slightly higher validation MAE<br>• Benefits limited because the architecture is not fully "self-normalizing" |
| Mixed Activations (ReLU → ELU → ReLU) | • Combines fast ReLU layers with stabilizing ELU layer | • | • Performs similarly to ReLU<br>• Does not outperform pure ELU<br>• Mixing activations can help in some architectures, but not significantly here |

## *Visualization of Learning Curves*

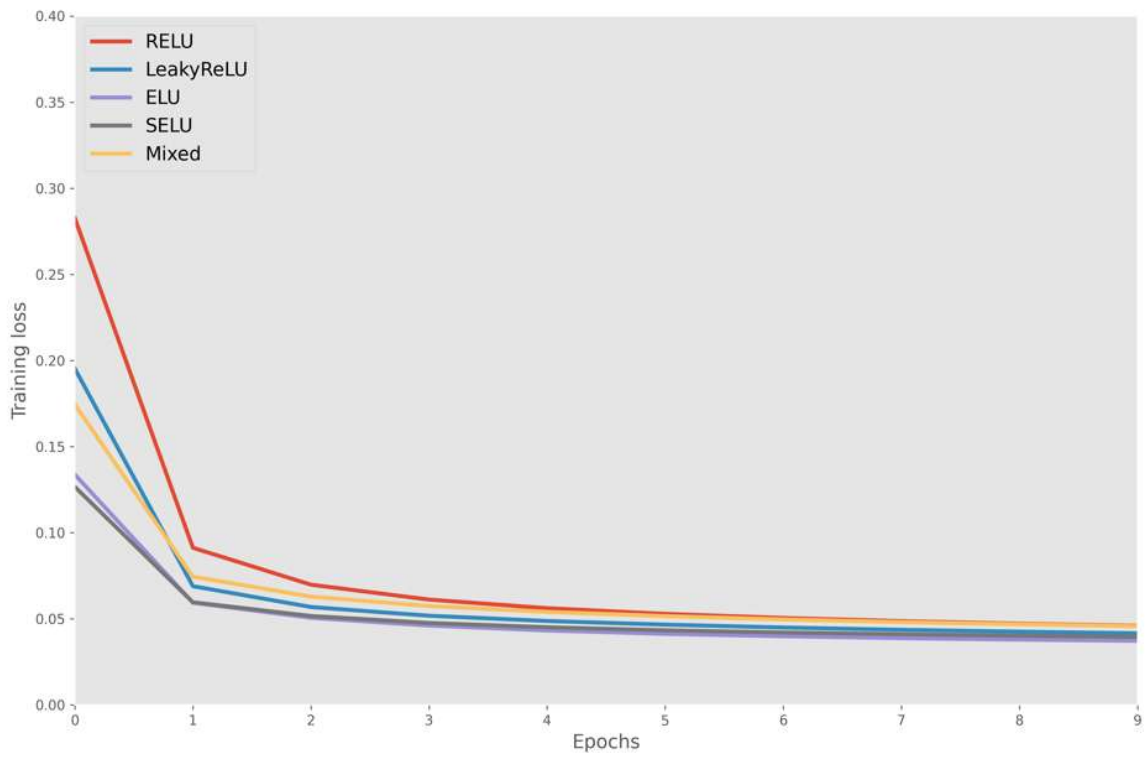



Figure 5: Training Loss

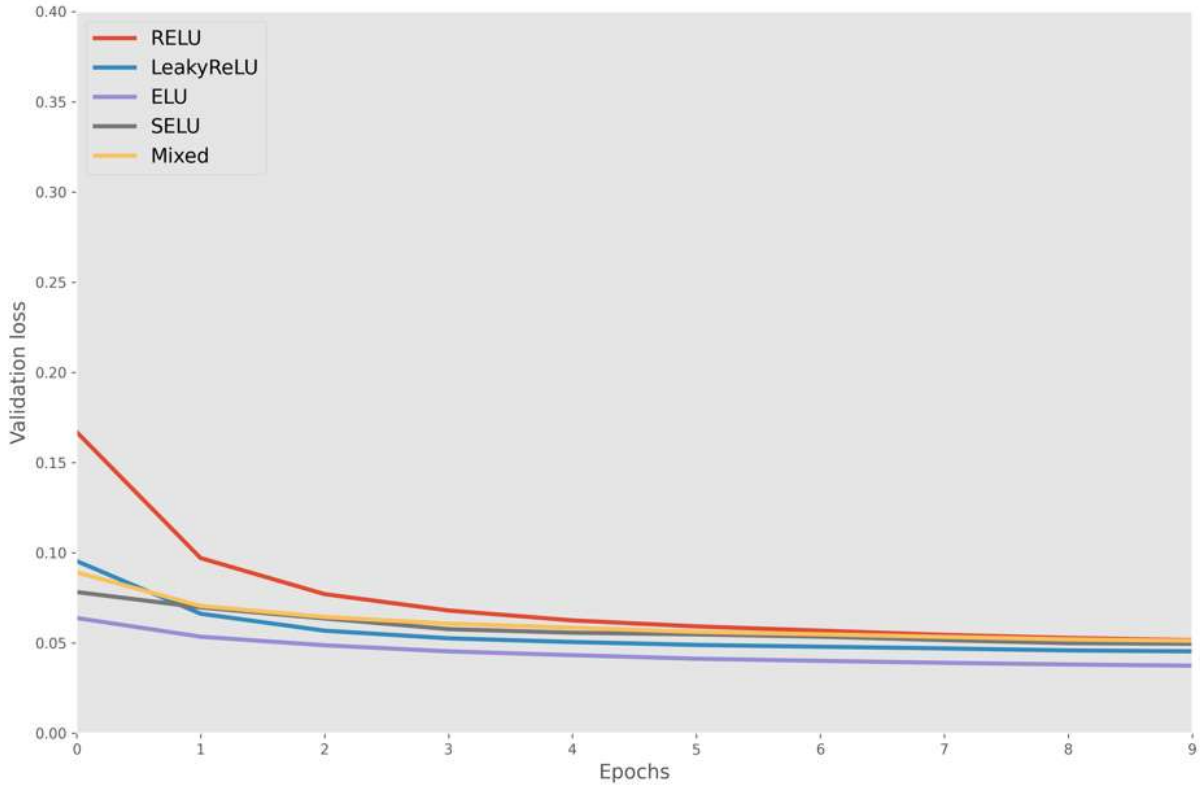



Figure 6: Validation Loss

## *Conclusion*

This experiment demonstrates that:

- Activation choice significantly impacts model performance.
- Functions that avoid zero-gradient regions (Leaky ReLU, ELU) consistently outperform standard ReLU.

## Optimizers in Deep Neural Networks

Optimizers are the **algorithms that adjust a neural network's weights during training** so the model can reduce its loss and learn patterns from data. They determine *how* the network moves through the loss landscape—how big each update is, how direction changes are handled, and how quickly or smoothly the model converges.

They are one of the most important choices in deep learning because they directly control training speed, stability, and final accuracy.

What an optimizer does

Every training step follows the same core idea:

1. The model makes a prediction.
2. The loss function measures how wrong the prediction is.
3. The optimizer uses the loss gradient to update the weights. Different optimizers change how this update is computed.

Why optimizers matter

Optimizers influence:

- **Training speed** (how fast the model learns)
- **Stability** (whether training oscillates or diverges)
- **Ability to escape poor local minima**
- **Sensitivity to learning rate**
- **Performance on noisy or complex data**

The same model can perform *very differently* depending on the optimizer.

## Comparing Optimizers in a Deep Neural Network

The experiment evaluates how different optimizers affect:

- Training speed
- Convergence stability
- Final model accuracy
- Generalization

Optimizers tested:

- **SGD-based:**
  - SGD
  - SGD + Momentum
  - Nesterov Accelerated Gradient
- **Adaptive Learning Rate:**
  - AdaGrad
  - RMSProp
- **Adam Family:**
  - Adam
  - Adamax
  - Nadam
  - AdamW

These represent the major optimization strategies used in modern deep learning.

SGD Family Results

| Optimizer | Final Val MAE |
|---|---|
| **SGD + Momentum** | **0.1221** |
| Nesterov | 0.1321 |
| SGD | 0.1441 |

Analysis

- Momentum significantly improves SGD by smoothing updates.
- Nesterov performs similarly but slightly worse in this setup.
- Plain SGD is slowest and least accurate.

AdaGrad and RMSProp

| Optimizer | Final Val MAE |
|---|---|
| **RMSProp** | **0.1012** |
| AdaGrad | 0.1558 |

Analysis

- RMSProp adapts learning rates effectively → fast, stable convergence.
- AdaGrad's learning rate decays too quickly → early stagnation.
- RMSProp outperforms all SGD variants.

Adam Family Results

| Optimizer | Final Val MAE |
|---|---|
| **AdamW** | **0.1035** |
| **Adam** | **0.1035** |
| Nadam | 0.1096 |
| Adamax | 0.1151 |

| Optimizer | Best Val MAE | Best Epoch |
|---|---|---|
| Adam | 0.1019 | 7 |
| AdamW | 0.1036 | 6 |

Analysis

- When using **best epoch**, **Adam slightly outperforms AdamW** on the dataset.
- When using **final epoch**, they tied at 0.1035.
- AdamW is more stable, but Adam reached a slightly better minimum.

Full Ranking of Optimizers

| Rank | Optimizer | Final Val MAE |
|---|---|---|
| **1** | **RMSProp** | **0.1012** |
| 2 | Adam / AdamW | 0.1035 |
| 3 | Nadam | 0.1096 |
| 4 | Adamax | 0.1151 |
| 5 | Momentum | 0.1221 |
| 6 | Nesterov | 0.1321 |
| 7 | SGD | 0.1441 |
| 8 | AdaGrad | 0.1558 |

## *Visualization of Learning Curves*

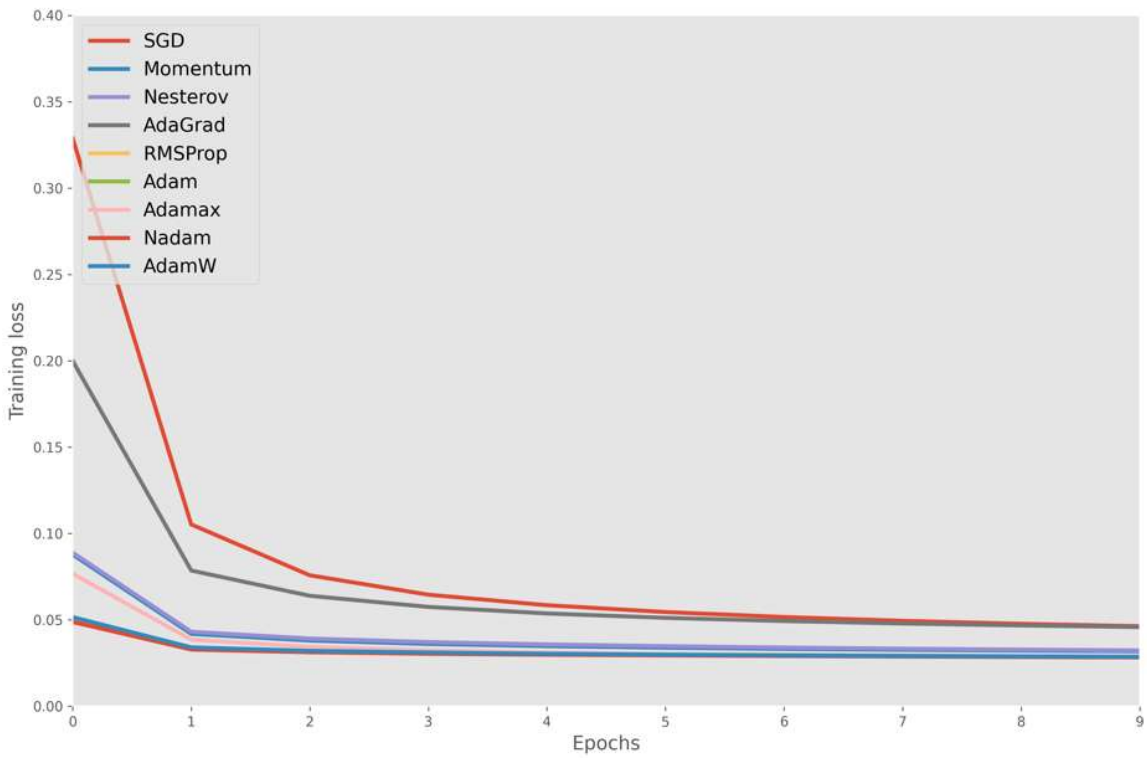



Figure 7: Training Loss

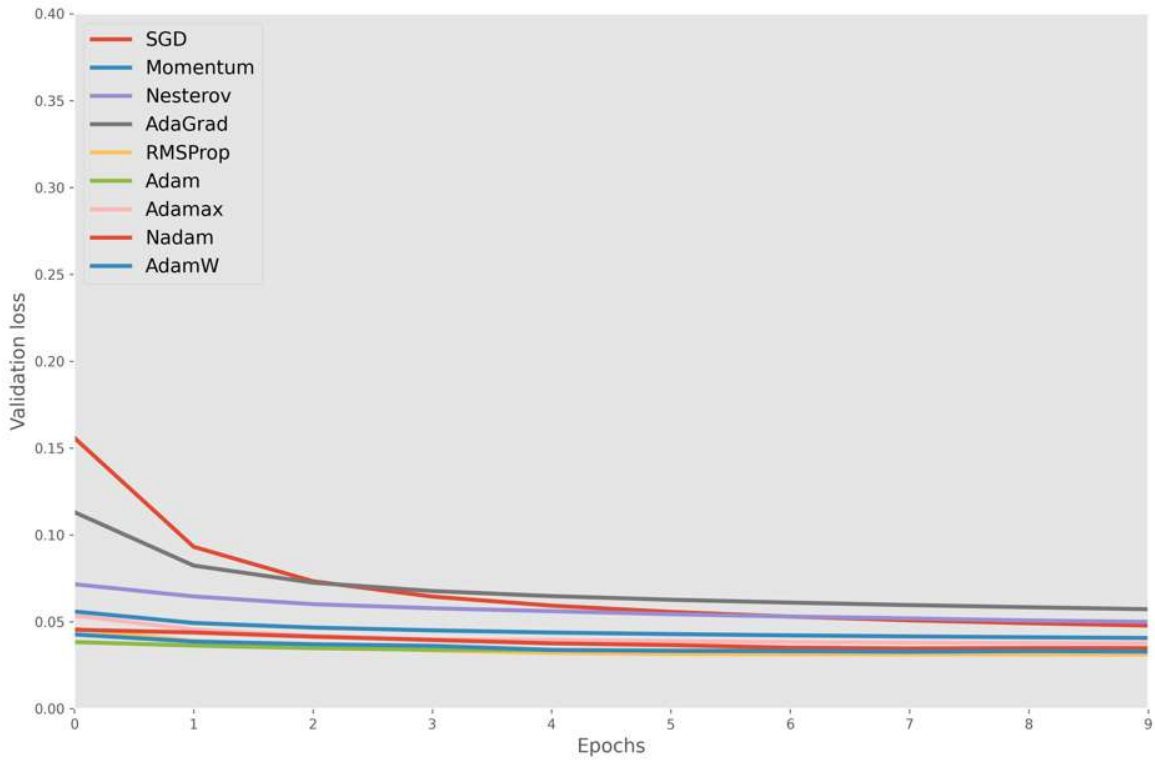



Figure 8: Validation Loss

## *Conclusion*

This experiment demonstrates that:

- Adaptive optimizers outperform SGD-based methods: RMSProp, AdamW, Adam, and Nadam converge faster and reach lower error.
- RMSProp is the best optimizer: per-parameter adaptive learning rates and exponential moving averages of squared gradients stabilize training and prevent overshooting.
- Adam remains an excellent default choice: fast, stable, and widely used.
- Momentum improves SGD significantly, but still cannot match adaptive methods.
- AdaGrad is not suitable for dense neural networks: its learning rate decays too quickly.

## Learning Rate Scheduling in Deep Neural Networks

A learning-rate schedule is a strategy for **changing the learning rate during training** instead of keeping it fixed. This matters because the ideal learning rate is *not* constant: models often learn best when they start with a larger step size (to explore the loss landscape) and gradually shrink it (to fine-tune around a minimum).

## Power Scheduling

Power scheduling (also called **polynomial decay**) is a learning-rate schedule where the learning rate decreases according to a **power-law function**.

The formula:

```
learning_rate = initial_learning_rate / (1 + step / decay_steps)**power
```

*Keras uses power = 1.*

Power scheduling gives:

- High learning rate early → fast learning
- Gradual decay → stable convergence
- Smooth or staircase options

I experimented optimizing neural network training with power scheduling (Inverse Time Decay) by comparing power scheduling smooth vs. staircase decay and optimized hyperparameters.

| Experiment | Hyperparameters | Results | Conclusion |
|---|---|---|---|
| Smooth Power Scheduling | Initial LR: **0.01**<br><br>Decay steps: **10,000**<br><br>Decay rate: **1.0**<br><br>Optimizer: **SGD**<br><br>Staircase: **False** | Validation MAE improved from **0.1467 → 0.1188**<br><br>Smooth, monotonic convergence<br><br>No overfitting | LR decay is **very slow**, but still effective<br>Training & validation MAE decrease steadily<br>Stable but not optimal — LR stays too high for too long |
| Staircase Power Scheduling | Initial LR: **0.01**<br><br>Decay steps: **10,000**<br><br>Decay rate: **1.0**<br><br>Optimizer: **SGD**<br><br>Staircase: **True** | Validation MAE improved from 0.1517 → 0.1135<br><br>Slightly better than smooth decay<br><br>Clear jumps in improvement after LR drops | Staircase decay performs **better** than smooth decay<br>Stepwise LR drops help escape shallow minima<br>Lower final MAE than smooth decay<br>Still no overfitting |
| Optimized Power Scheduling | Initial LR: **0.03**<br><br>Decay steps: **2,000**<br><br>Decay rate: **2.0**<br><br>Optimizer: **SGD** | Validation MAE improved **0.1070 → 0.0950**<br>Best performance across all experiments<br>Fast early | Higher initial LR → faster early learning<br>Smaller decay steps → LR drops sooner<br>Larger decay rate → stronger decay<br>Momentum + Nesterov → smoother, faster |

| | | | |
|---|---|---|---|
| | Staircase: **True** | convergence, stable late training | convergence<br>Achieves **lowest MAE: 0.0950** |

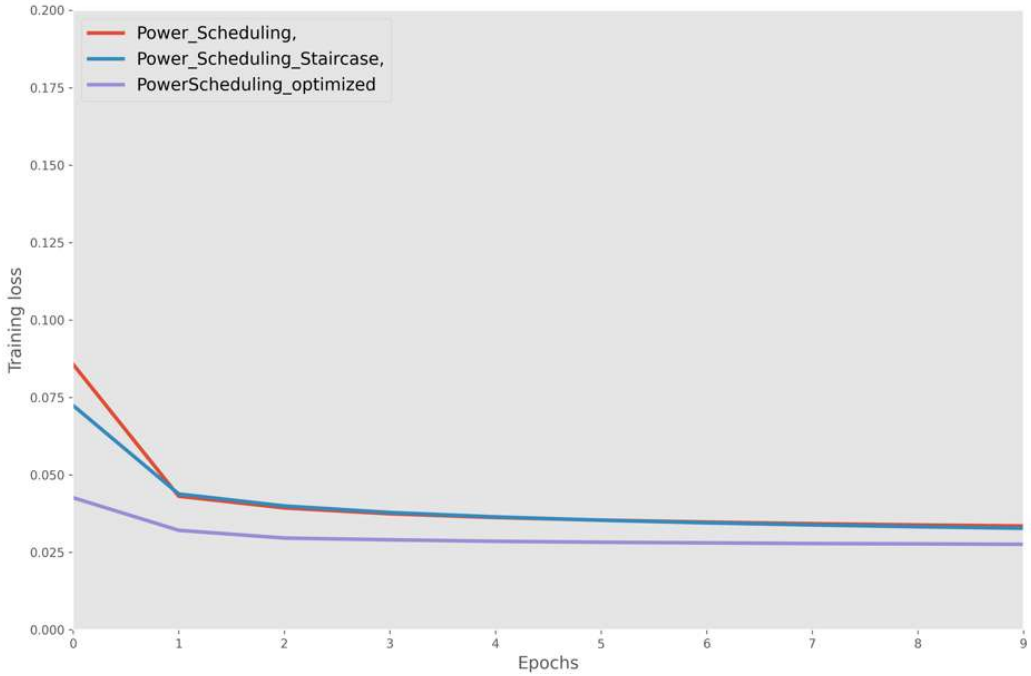



Figure 9: Training Loss

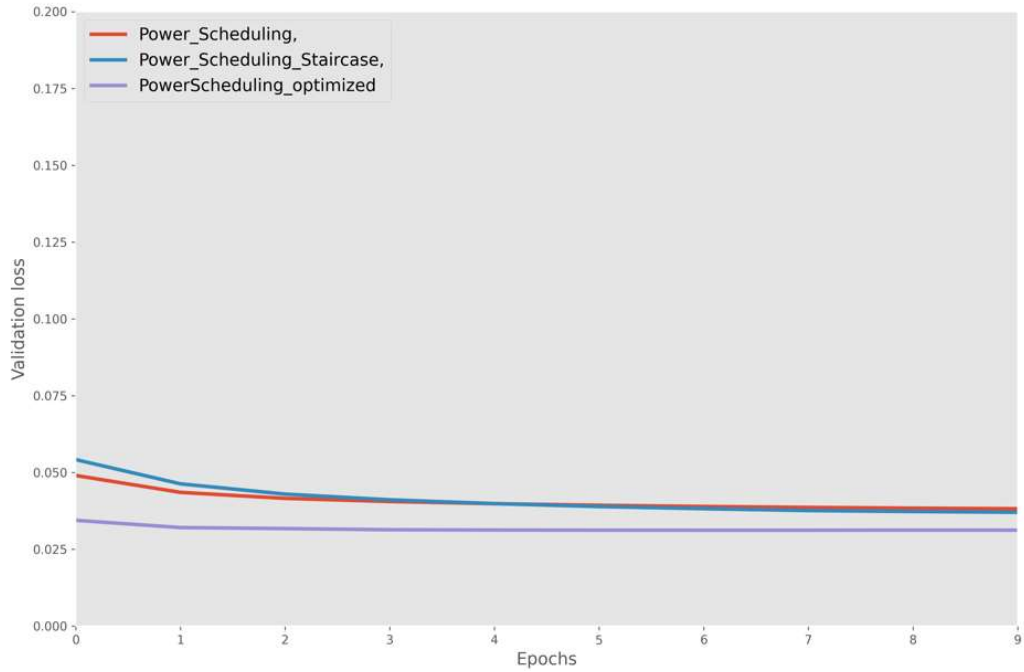



Figure 10: Validation Loss

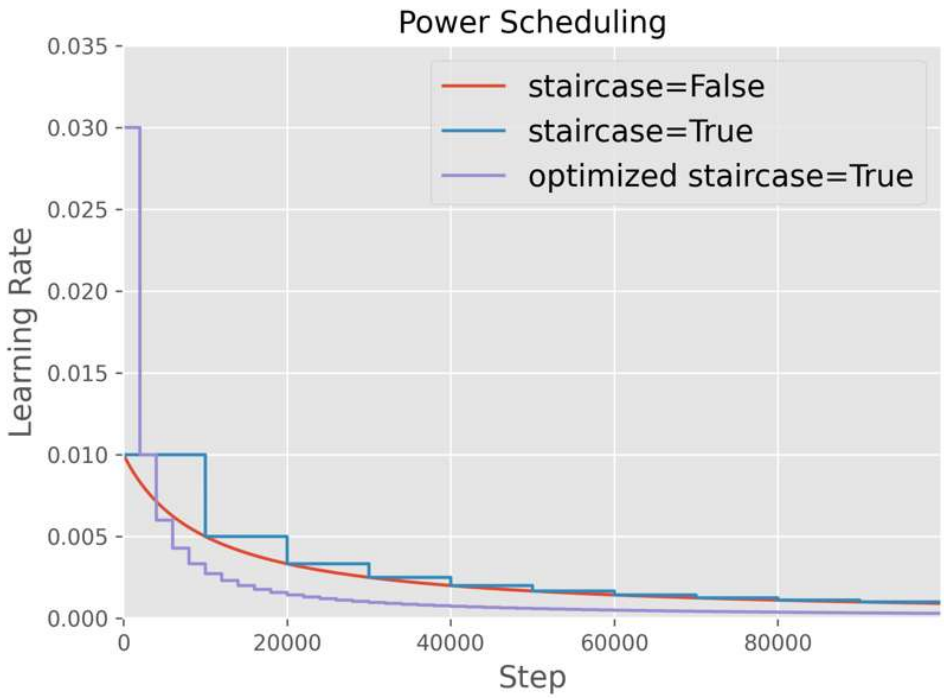



Figure 11: Power Scheduling

## Exponential Scheduling

Exponential scheduling is a learning-rate decay strategy where the learning rate is reduced **multiplicatively** after each epoch (or step). Instead of dropping the learning rate suddenly at fixed milestones, it shrinks smoothly and continuously.

The formula:

learning_rate = initial_learning_rate * decay_rate ** (step / decay_steps)

Exponential scheduling gives a **high learning rate early** (fast exploration) and a **small learning rate later** (fine-tuning). This helps:

- converge faster
- avoid overshooting minima
- stabilize training in later epochs
- reduce oscillation around the optimum

It is smoother than step decay and simpler than cosine or cyclical schedules.

I experimented optimizing neural network training with exponential learning rate scheduling by comparing exponential learning rate without staircase, with staircase, and optimized LR schedules.

| Experiment | Hyperparameters | Results | Conclusion |
|---|---|---|---|
| Smooth Exponential Decay | Initial LR: **0.01**<br><br>Decay steps: **20,000**<br><br>Decay rate: **0.1**<br><br>Optimizer: **SGD**<br><br>Staircase: **False** | Validation MAE improved from 0**.1577 → 0.1221**<br><br>Smooth, consistent convergence<br><br>No signs of overfitting | Smooth decay provides stable refinement across all 10 epochs.<br><br>Training and validation curves decrease steadily<br><br>Validation MAE improves every epoch<br><br>LR decay helps prevent plateauing but converges slowly |
| Staircase Exponential Decay | Initial LR: **0.01**<br><br>Decay steps: **20,000**<br><br>Decay rate: **0.1**<br><br>Optimizer: **SGD**<br><br>Staircase: **True** | Validation MAE improved from **0.1501 → 0.1120**<br><br>Better than smooth decay<br><br>Stepwise LR drops help the optimizer "reset" momentum | Staircase decay converges faster and to a lower MAE<br>LR jumps create sharper improvements after each drop<br>Validation MAE consistently lower than smooth decay<br>Still no overfitting |
| Optimized LR Schedule | Initial LR: **0.03**<br><br>Decay steps: **2,000**<br><br>Decay rate: **0.5**<br><br>Optimizer: **SGD + Momentum (0.9) + Nesterov**<br><br>Staircase: **True** | Validation MAE improved dramatically: **0.1069 → 0.0985**<br><br>Best performance across all experiments<br><br>Training stabilized by epoch 3–4<br><br>No overfitting observed | Higher initial LR → faster early learning<br>Faster decay → stable fine-tuning in later epochs<br>Momentum + Nesterov → smoother, faster convergence<br>Combining exponential decay + momentum + Nesterov yields the strongest results |

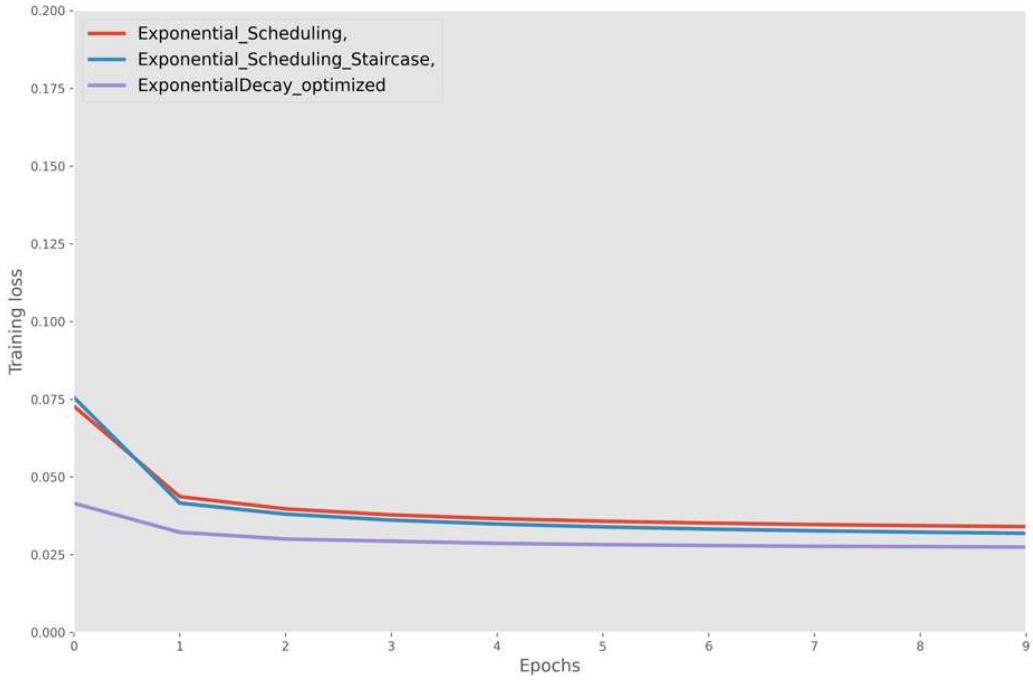



Figure 12: Training Loss

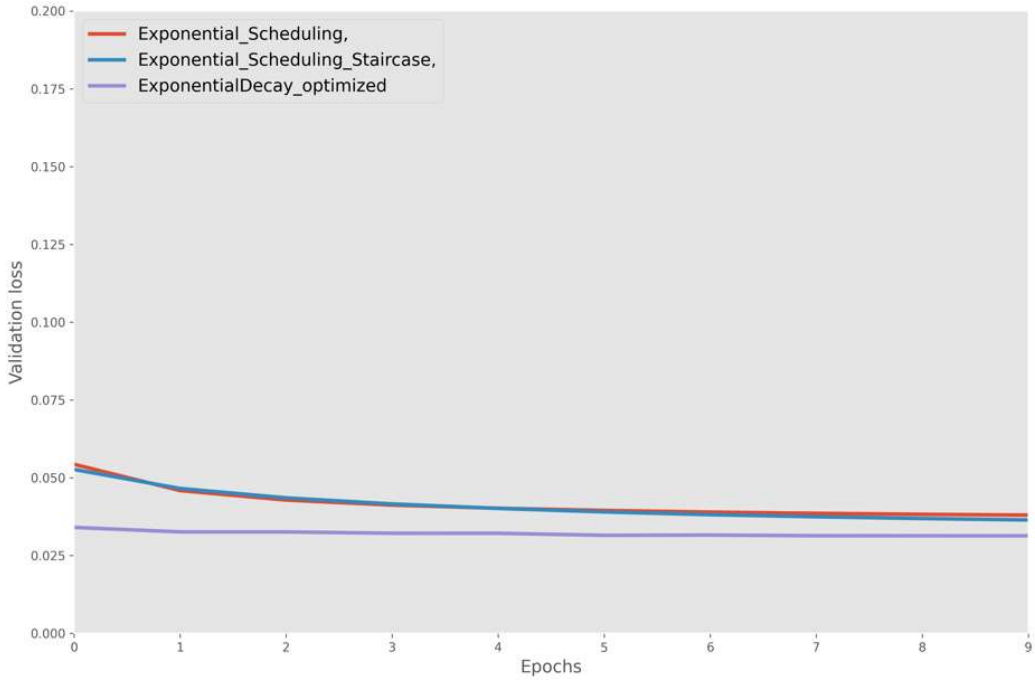



Figure 13: Validation Loss

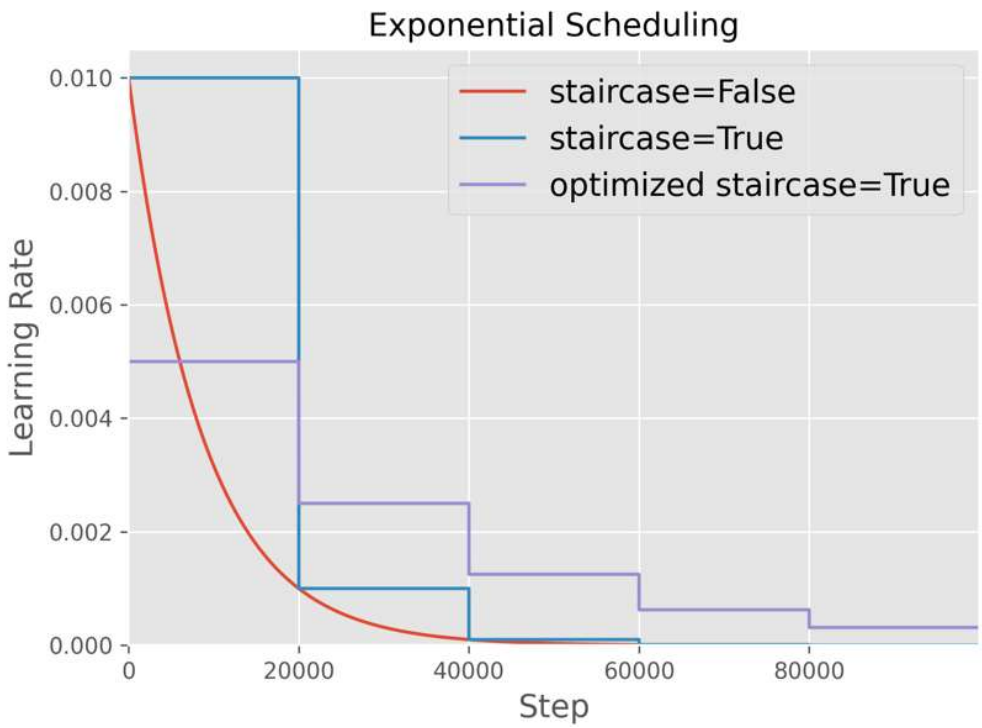



Figure 14: Exponential Scheduling

## Early Stopping

Early Stopping is a training technique where the model **stops training automatically when it stops improving on the validation set**, ensuring we get the best weights without wasting time or overfitting.

Instead of training for a fixed number of epochs (like 10, 20, or 50), we let the model decide when it has learned enough.

During training, we monitor a metric (usually **validation loss**). If the metric **doesn't improve for several epochs**, training stops. The model **restores the best weights**, not the final ones.

Early Stopping:

- Prevents overfitting. If the model starts memorizing noise, validation loss will stop improving. Early Stopping halts training before that happens.
- Saves time. We don't waste epochs after the model has already reached its best performance.
- Automatically finds the best epoch. No need to guess whether 10, 20, or 50 epochs is best.

Early Stopping works best when:

- validation loss starts to flatten
- training loss keeps decreasing but validation loss doesn't
- we are not sure how many epochs to train
- we want the best possible generalization

## Improving Neural Network Training with EarlyStopping

The purpose of the experiment is to train a regression neural network on tabular data and compare **baseline training** (fixed 50 epochs) with **EarlyStopping**. Will measure improvements in: validation loss, training efficiency and generalization.

| Experiment | Hyperparameters | Results | Conclusion |
|---|---|---|---|
| Baseline setup | Optimizer Adamax<br><br>No callback during training<br><br>50-epoch training window | Final Validation Loss: **0.03383**<br><br>Strong improvement in first ~10 epochs<br><br>After epoch 10, validation loss barely changes (plateau)<br><br>Model continues training unnecessarily | The model does **not** overfit, but continues training long after improvements stop<br>Many epochs are wasted with no meaningful gain<br>Training is inefficient without a stopping criterion |
| EarlyStopping | Optimizer Adamax<br><br>EarlyStopping with<br><br>• monitor='val_loss' → best metric for regression<br>• patience=3 → allows small fluctuations<br>• restore_best_weights =True → ensures best epoch is used | Best validation loss: **0.03202**<br>Training stopped at **epoch 29**<br>Validation loss improved steadily until plateauing | **Improvement over baseline:**<br>• Baseline final val_loss: 0.03383<br>• EarlyStopping best val_loss: 0.03202<br>**• ≈ 5.3% lower validation loss**<br><br>EarlyStopping prevents wasted epochs, restores the best weights, and produces a better-generalizing model. |

## *Learning Curve Comparison*

**Baseline**

- Runs full 50 epochs
- Validation loss plateaus after epoch ~10
- Later epochs add no benefit

**EarlyStopping**

- Stops at epoch 29
- Captures the best validation performance
- Avoids unnecessary training

## *Why EarlyStopping Helped*

- The model continued making meaningful improvements up to **around epoch 29**
- After that point, validation loss **flattened and stopped improving**
- EarlyStopping provided several advantages:
    - **Avoids late-epoch noise** once the model reaches its optimal region
    - **Prevents drift** away from the best weights by restoring the strongest checkpoint
    - **Reduces unnecessary training**, cutting the run from 50 epochs to 29
    - **Improves generalization**, achieving a lower validation loss than the baseline

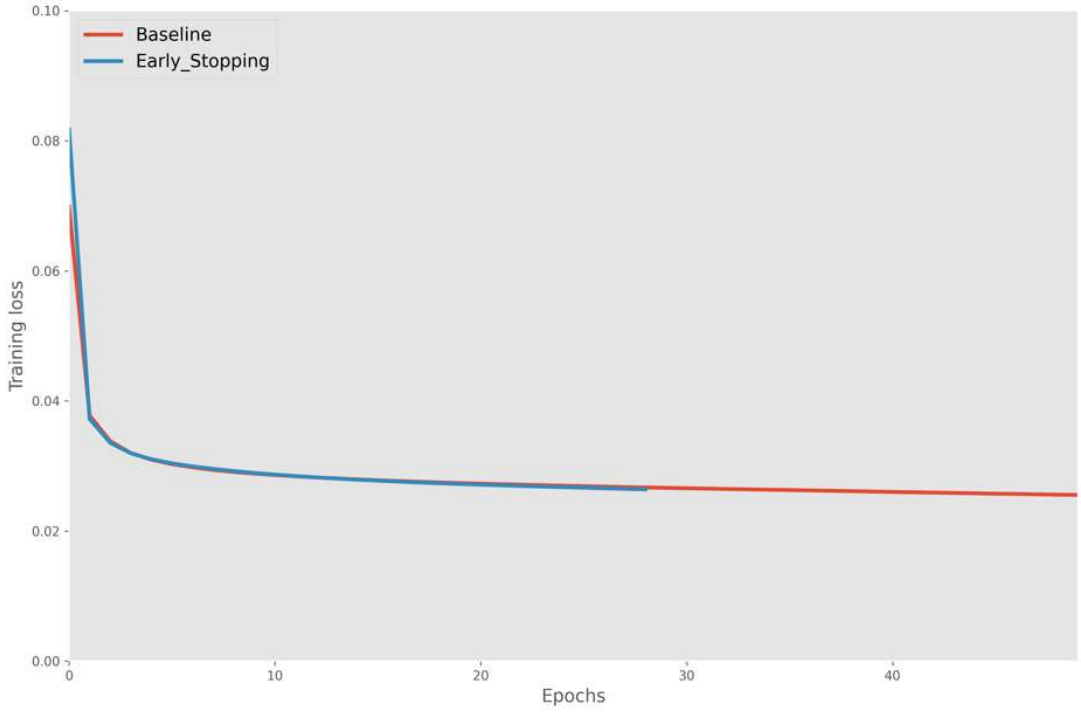


Figure 15: Training Loss

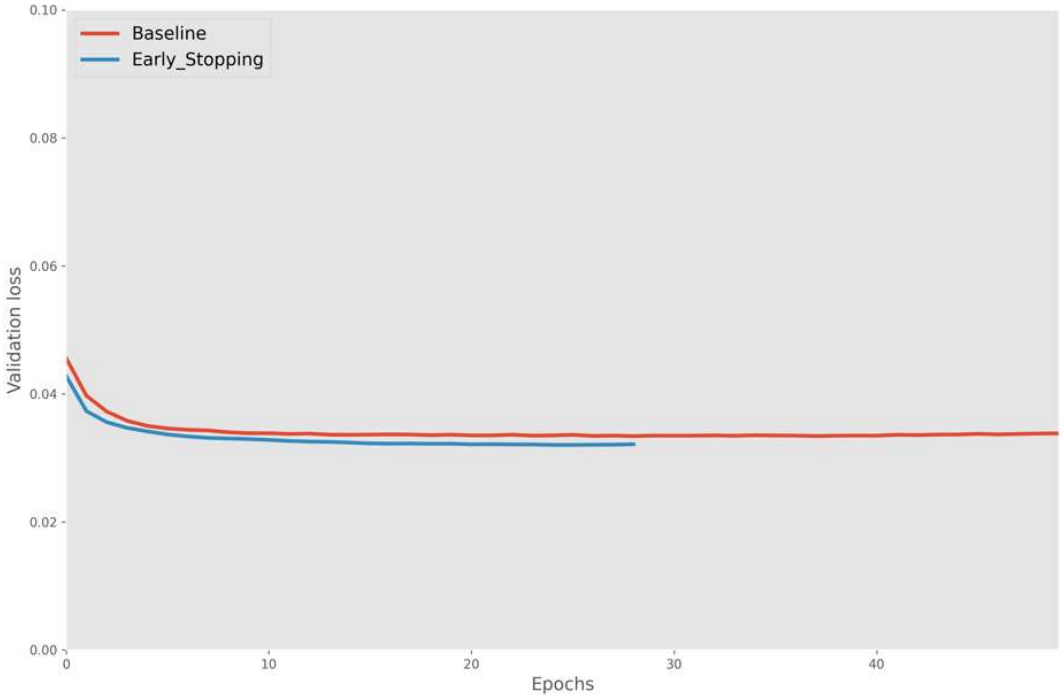



Figure 16: Validation Loss

## Conclusion

Over the course of the project, I explored how different training strategies influence the performance and stability of a deep neural network. Starting from a simple baseline, I progressively introduced more advanced optimization techniques: weights initialization, activation functions, optimizers, learning-rate scheduling, and finally EarlyStopping—to understand how each component shapes the learning dynamics of a model.

Across all experiments, one theme consistently emerged: **small, well-targeted adjustments can have a large impact on model quality**. Whether it was tuning the learning rate, choosing the right optimizer, or adding a callback, each refinement contributed to a more stable, efficient, and accurate model.

Ultimately, this work demonstrates that optimizing deep neural networks is not just about building bigger architectures—it's about understanding the training process itself. With thoughtful experimentation and the right tools, even a modest model can achieve strong, reliable performance.